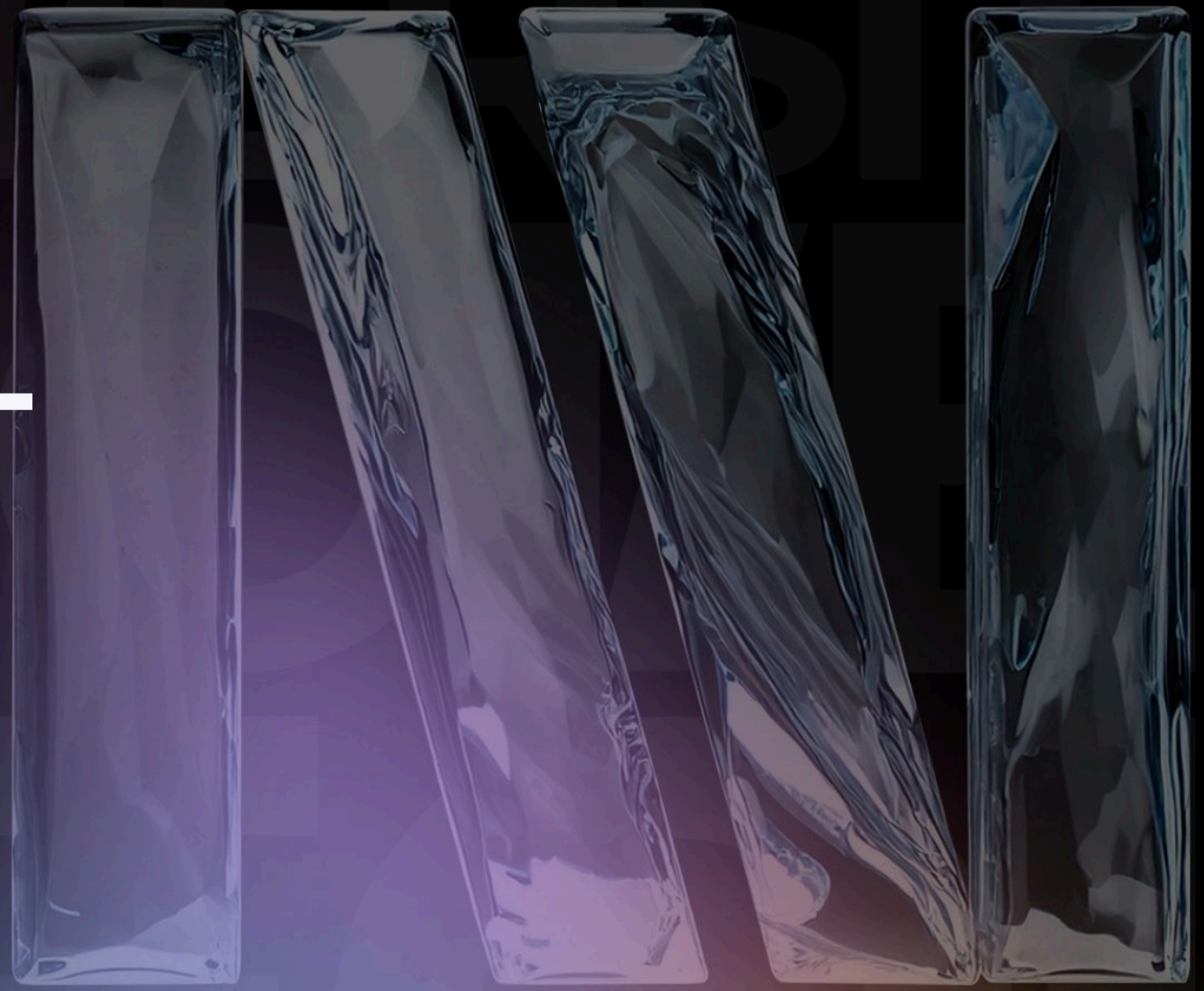
ПРИВІТ

Скоро починаємо



ПРАВИЛА

Правила використання чату

01 Будьте активними — відповідайте на питання викладача

02 Питання не за темою заняття в чат **не** пишемо

03 Знаєте відповідь на питання в чаті — сміливо відповідайте. Працюємо як команда!

04 Про технічні проблеми пишіть одразу, під час заняття

Важливо!

Якщо повітряна тривога — йдемо в укриття.

Запис заняття та презентація обов’язково будуть доступні в LMS після заняття.

ВИКЛАДАЧ

Олег Андрус

Викладач курсу «Python Core» у GoIT Neoversity

- Магістр комп’ютерних наук і математики (ЛНУ ім. І. Франка)
- 7 років у розробці, зараз — Senior Python Developer у Tallium
- Досвід: Levi9, GlobalLogic
- Викладаю в Neoversity з 2023 року



NEOVERSITY



Заняття 1

Модуль 1

ЗНАЙОМСТВО З PYTHON. ЯК ПРАЦЮЄ МОВА І ЇЇ ПРАВИЛА

/ PYTHON CORE

ПРО КУРС

Про що цей курс?

Python Programming: Foundations and Best Practices — ти поступово розберешся:

01

Основи мови

типи даних, керуючі конструкції, винятки, функції та структури даних

02

Робота з даними

рядки, файли, серіалізація та копіювання об’єктів

03

Класи й пакети

класи, «магічні» методи, власні пакети та віртуальне оточення

Фінал

Командний проєкт, фінальне тестування і Демо День.

ФОРМАТ НАВЧАННЯ

Python Core

Тривалість курсу

6 тижнів

+ тиждень 7 — дипломний модуль

Живі заняття

2 на тиждень

з викладачем

Час занять

19:30 – 21:30

Перерва

10 хв

ПРОГРАМА КУРСУ

12 модулів за 7 тижнів

Тиждень 1 сьогодні

Модуль 1. Типи даних у Python

Модуль 2. Керуючі конструкції. Винятки

Тиждень 2

Модуль 3. Функції

Модуль 4. Структури даних

Наукова публікація 1

Тиждень 3

Модуль 5. Просунута робота з рядками

Модуль 6. Робота з файлами

Тиждень 4

Модуль 7. Створення та встановлення власних пакетів, віртуальне оточення

Модуль 8. Можливості деяких вбудованих пакетів Python

Тиждень 5

Модуль 9. Функції (декоратори, замикання)

Модуль 10. Основи роботи з класами

Наукова публікація 2

Тиждень 6

Модуль 11. «Магічні» методи

Модуль 12. Серіалізація та копіювання об’єктів

Тиждень 7 · Дипломний модуль

Командний проєкт

Фінальне тестування

Демо День

НАВЧАННЯ

Обов’язкові навчальні ресурси

01	Конспект	2 на тиждень
02	Живі заняття з викладачем	2 на тиждень
03	Домашні завдання: автоперевірки та ДЗ	
04	Наукові публікації та написання короткого рев’ю	2 на курс
05	Фінальне тестування	1 на курс
06	Фінальний командний проєкт	
07	Демо День	

ПОРАДИ

Рекомендації до навчання

01 Переглядай матеріали до початку заняття

02 Плануй свій час і не чекай, коли горітимуть дедлайни

03 Є питання? Конспект + пошук + ментор чи заняття

04 Один у полі **не** воїн — працюй із групою

05 Не бійся ставити запитання і шукати відповіді

06 Пиши код руками — копіювання не вчить

Не встиг підготуватися — все одно приходь, а після заняття прочитай конспект і переглянь запис.

ПОЧИНАЄМО

НАВЧАННЯ!

ПЛАН ЗАНЯТТЯ

Сьогодні ми

01 Python як мова: історія, версії, застосування, встановлення

02 Як працює Python: компілятор, інтерпретатор, пам’ять, типи

03 Правила гри: ключові слова, Zen of Python, PEP 8

01

Python як мова



Автор · назва · версії · родовід · застосування · встановлення

ХТО СТВОРИВ

Гвідо ван Россум

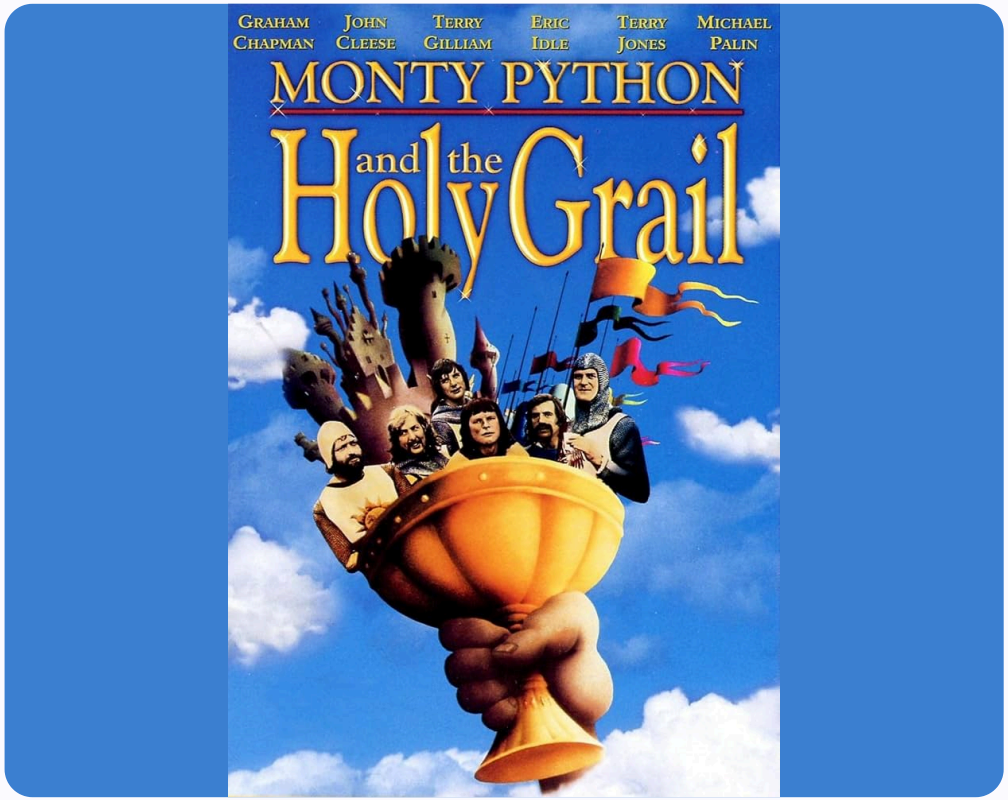
Нідерландський програміст. До 2018 року — «великодушний довічний диктатор» Python.

1989	Почав писати Python як хобі на різдвяних канікулах
1991	Перша публічна версія — Python 0.9.0
2005+	Працював у Google, Dropbox, Microsoft



ЧОМУ PYTHON

Назва — від комедійного шоу, а не від змії



Monty Python

Гвідо читав сценарії шоу «Літаючий цирк Монті Пайтона» і хотів коротку, трохи загадкову назву.

Змія — пізніше

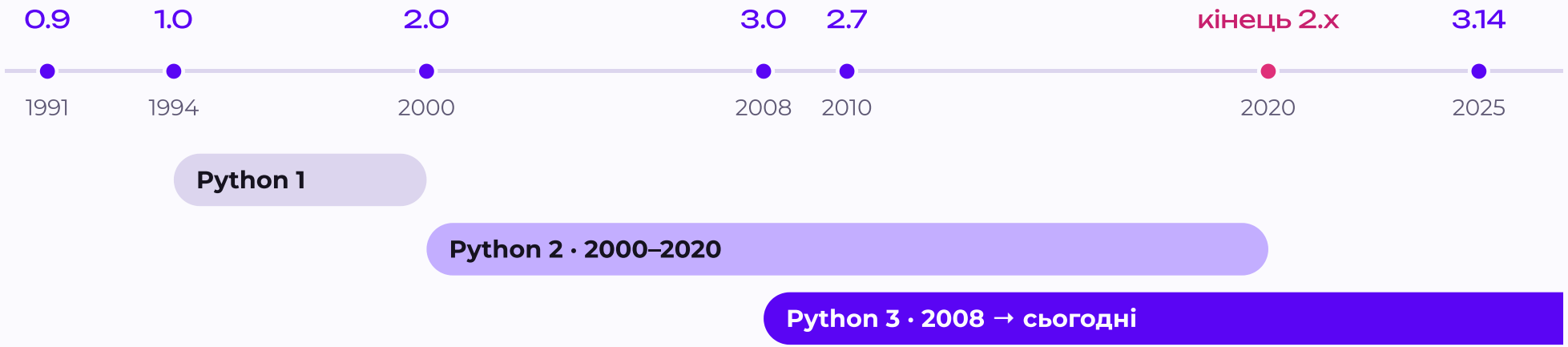
Логотип зі зміями з'явився згодом як гра слів.

Відсилки досі живі

У прикладах замість foo і bar пишуть `spam` та `eggs` — це скетч із шоу.

ВЕРСІЇ

Як змінювався Python



2 → 3 несумісні

`print "hi"` → `print("hi")`

Ритм релізів

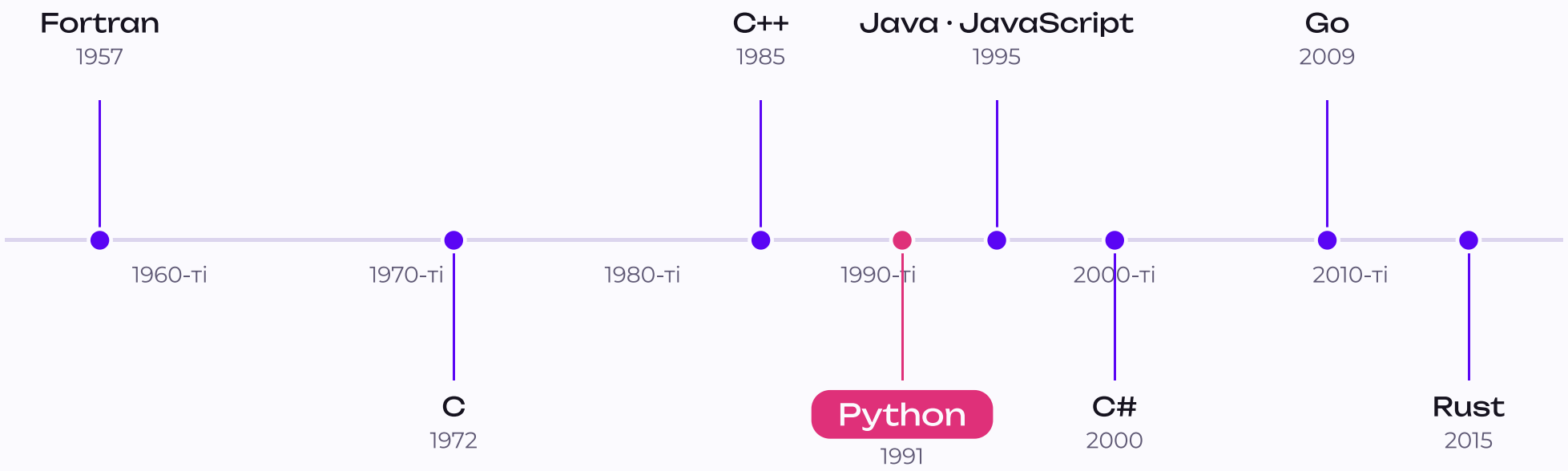
Нова версія щороку в жовтні, підтримка — 5 років

Ми вчимо

Лише Python 3 — версію 3.12 або новішу

ХРОНОЛОГІЯ

Python серед інших мов



Python з’явився раніше за Java та JavaScript, але став найпопулярнішим у 2010-х — разом із розквітом даних і ШІ.

РОДОВІД

Від кого Python узяв ідеї — і кому передав

ВПЛИНУЛИ НА PYTHON

ABC	1987	відступи замість дужок
Modula-3	1988	модулі та винятки
C	1972	оператори; CPython написаний на C
Lisp	1958	lambda, map, filter
Haskell	1990	генератори списків



PYTHON ВПЛИНУВ НА

Ruby	1995
Go	2009
Julia	2012
Swift	2014
Mojo	2023

ЗАСТОСУВАННЯ

Де живе Python

01

Веб-розробка

`Django · FastAPI`

Instagram, Pinterest

02

Дані та аналітика

`pandas · Jupyter`

Netflix, Spotify

03

Штучний інтелект

`PyTorch · TensorFlow`

ChatGPT, рекомендації

04

Автоматизація

`скрипти · боти`

рутина без людини

05

Наука

`NumPy · SciPy`

NASA, CERN

06

Тестування та DevOps

`pytest · Ansible`

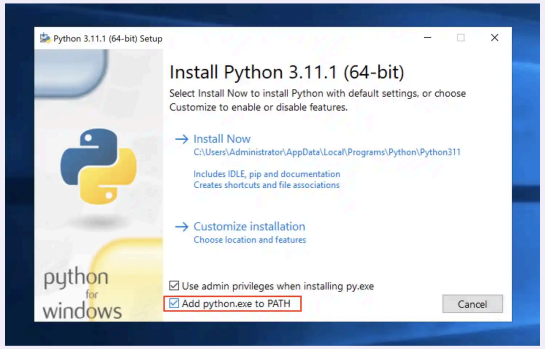
QA- та DevOps-команди

ВСТАНОВЛЕННЯ

Встановлюємо Python

Windows

1. python.org/downloads → Download
2. Відкрий інсталятор
3. **Галочка «Add python.exe to PATH»**
4. Install Now



macOS

1. python.org/downloads → .pkg
2. Встанови як звичайну програму

або через Homebrew:

```
brew install python
```

Linux

1. Часто вже встановлений
2. Якщо ні — менеджер пакетів

```
sudo apt update
sudo apt install python3
```

ПЕРЕВІРКА

Перевіряємо, що все працює

Термінал

```
$ python --version
Python 3.14.0

$ python
Python 3.14.0 … on win32
>>> 2 + 2
4
>>> print("Привіт!")
Привіт!
>>> exit()
```

macOS / Linux

Пиши `python3` замість `python`

>>> — інтерактивний режим

REPL: пишеш рядок — одразу бачиш результат

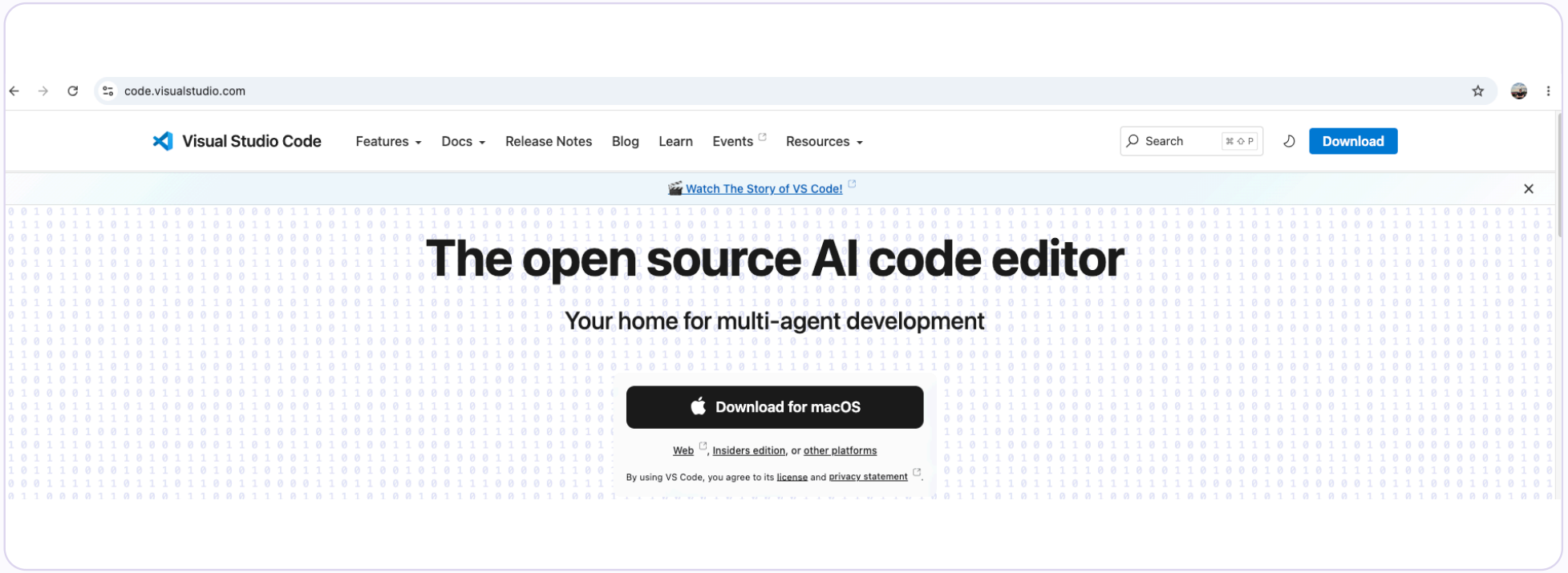
«python is not recognized»

Python не додано в PATH — перевстанови з галочкою

РЕДАКТОР КОДУ

Встановлюємо VS Code

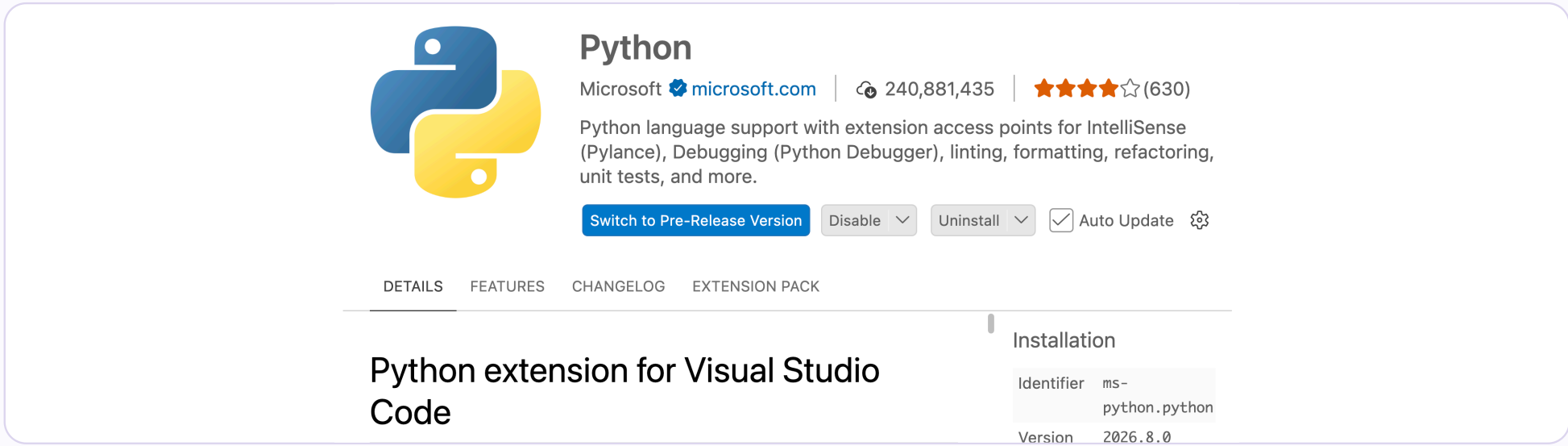
1 **Завантаж** code.visualstudio.com → Download → встанови як звичайну програму



РЕДАКТОР КОДУ

Додаємо розширення Python

2 **Розширення** Extensions (Ctrl+Shift+X) → «Python» від Microsoft → Install



`Ctrl + ``
відкрити термінал

`Ctrl + Shift + P`
палітра команд

`Ctrl + /`
закоментувати рядок

`Ctrl + S`
зберегти файл

02

Як працює Python

Перекладач · компілятор · байт-код · пам’ять · типи

АНАЛОГІЯ

Уяви двох перекладачів

Письмовий перекладач

перекладає всю книгу заздалегідь

Книга
англійською

→

Переклад
усієї книги — тижні роботи

→

Готова книга
читаєш швидко, без перекладача

Синхронний перекладач

перекладає на ходу, фразу за фразою

Фраза
мовця

→

Переклад
цієї фрази — одразу

→

Слухач
розуміє; наступна фраза...

ТЕПЕР ТЕРМІНАМИ

Компілятор та інтерпретатор

Компілятор

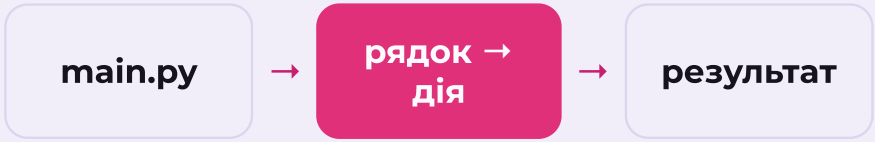
= письмовий перекладач · C, C++, Go, Rust



Перекладає всю програму до запуску.
Працює швидко. Помилки — до запуску.

Інтерпретатор

= синхронний перекладач · Python, JavaScript



Виконує код рядок за рядком.
Легко експериментувати. Помилка — під час запуску.

Наслідок для Python

Якщо помилка в рядку 5, рядки 1–4 вже встигнуть виконатися.

ПОДИВИМОСЬ У ДІЇ

Той самий код — два способи виконання

Компілятор спершу весь файл

```
1  name = "Оля"
2  print("Привіт,", name)
3  print(2 + 3)
4  print("Готово")
```

Перекладаю весь файл...

РЕЗУЛЬТАТ

Інтерпретатор рядок за рядком

```
1  name = "Оля"
2  print("Привіт,", name)
3  print(2 + 3)
4  print("Готово")
```

Прочитав рядок → одразу виконав

РЕЗУЛЬТАТ

ЗА ЛАШТУНКАМИ

Що відбувається, коли ти запускаєш python main.py



ТВІЙ КОД

```
x = 2
print(x + 3)
```

БАЙТ-КОД (import dis; dis.dis(...))

```
LOAD_CONST     2
STORE_NAME     x
LOAD_NAME      print
LOAD_NAME      x
LOAD_CONST     3
BINARY_OP      +
CALL           1
```

ПАМ’ЯТЬ

Де живуть дані програми



RAM — оперативна пам’ять

Швидка, але тимчасова: вимкнули програму — дані зникли

Комірки з адресами

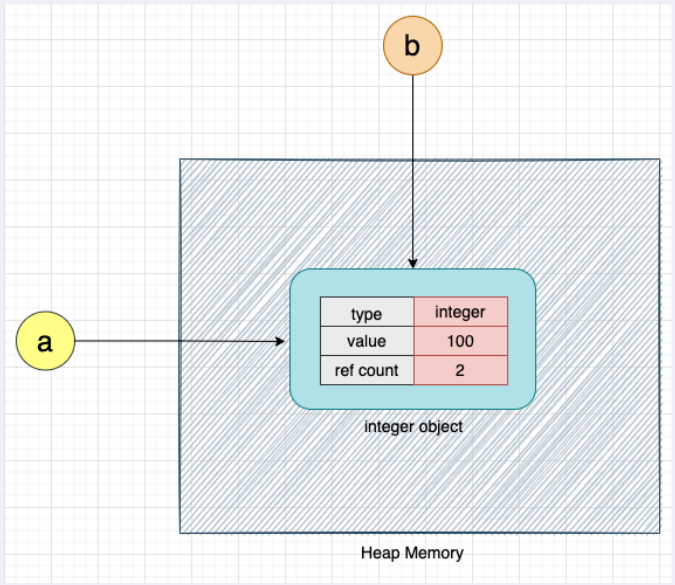
Кожна «банка» має номер — адресу. Так Python знаходить дані

Об’єкти

Кожне значення — число, рядок, список — це об’єкт у пам’яті

ЗМІННІ

Змінна — це ярлик, що вказує на об’єкт




```
a = [1, 2, 3]
b = a           # той самий об’єкт
print(id(a))    # 43621875
print(id(b))    # 43621875
print(a is b)   # True
```

`id()` — адреса об’єкта

Однакові id — це один і той самий об’єкт у пам’яті

ТИП ДАНИХ

Тип — це «вид» значення

Тип каже Python, **скільки місця** займає значення і **що з ним можна робити**: числа — додавати, текст — склеювати.

`42` `int`	`3.14` `float`	`"Київ"` `str`	`True` `bool`
`None` `NoneType`	`[1, 2]` `list`	`(1, 2)` `tuple`	`{"a": 1}` `dict`

```
type(42)        # <class 'int'>
type("42")      # <class 'str'>
```

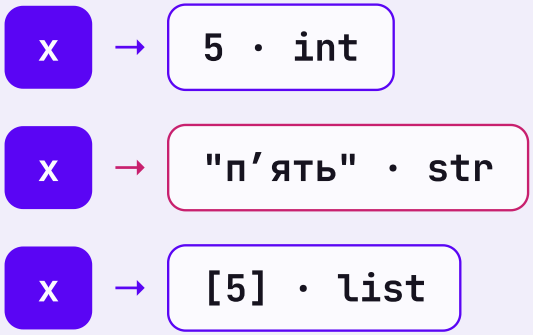
```
2 + 2           # 4
"2" + "2"       # '22'
```

ДИНАМІЧНА ТИПІЗАЦІЯ

Тип належить значенню, а не змінній

PYTHON

```
x = 5
print(type(x))   # int
x = "п’ять"
print(type(x))   # str
x = [5]
print(type(x))   # list
```



Той самий ярлик — щоразу інший об’єкт

Статична (Java, C++)

int x = 5; — тип оголошується, змінити не можна

Але сувора!

"5" + 5 → TypeError. Python не змішує типи сам

РІВНІ МОВ

Високий рівень — ближче до людини

Високий рівень
Python

```
print(2 + 3)
```

Середній рівень
C

```
printf("%d", 2 + 3);
```

Низький рівень
Асемблер

```
mov eax, 2   add eax, 3   call print
```

Машинний код
Процесор

```
10111000 00000010 00000101 …
```

Python сам керує пам’яттю й апаратурою — ти думаєш про задачу, а не про процесор.

ПІДСУМОК РОЗДІЛУ

Python — це високорівнева, інтерпретована мова програмування з динамічною типізацією

високорівнева
пишемо майже людською мовою

інтерпретована
виконується рядок за рядком через PVM

динамічна типізація
тип визначається під час виконання

03

Правила гри

Ключові слова · Zen of Python · PEP · PEP 8

ГЛОСАРІЙ

35 зарезервованих слів Python

False	None	True	and	as	assert	async
await	break	class	continue	def	del	elif
else	except	finally	for	from	global	if
import	in	is	lambda	nonlocal	not	or
pass	raise	return	try	while	with	yield

значення · логіка · керування потоком · функції й класи · інше — вивчимо пізніше

```
class = 5 → SyntaxError
```

PEP 20

Zen of Python — філософія мови

Термінал

```
>>> import this
The Zen of Python,
by Tim Peters
…
```

19 принципів від Тіма Пітерса (1999). Сховані прямо в Python — спробуй сам.

Beautiful is better than ugly.
Гарне краще за потворне.

Explicit is better than implicit.
Явне краще за неявне.

Simple is better than complex.
Просте краще за складне.

Readability counts.
Читабельність має значення.

Errors should never pass silently.
Помилки не мають замовчуватися.

There should be one obvious way to do it.
Має бути один очевидний спосіб.

PEP

Python Enhancement Proposal

Пропозиція щодо покращення Python: нова можливість, правило чи стандарт. Будь-хто може запропонувати — спільнота обговорює і приймає.

Ідея
будь-хто з спільноти

→

Чернетка PEP
опис і аргументи

→

Обговорення
python.org, discuss

→

Рішення
Steering Council

→

Python
нова версія

PEP 8
Стиль коду
як оформлювати код

PEP 20
Zen of Python
філософія мови

PEP 257
Docstrings
як документувати

PEP 484
Type hints
анотації типів

PEP 8 · ІМЕНА

Як правильно називати

Що називаємо	Стиль	Добре	Погано
Змінна, функція	`snake_case`	`user_name, get_total()`	`userName`
Константа	`UPPER_SNAKE_CASE`	`MAX_SIZE = 100`	`maxSize`
Клас	`PascalCase`	`UserProfile`	`user_profile`
Модуль (файл)	`lowercase`	`utils.py, my_module.py`	`MyModule.py`
«Внутрішнє»	`_underscore`	`_cache`	—

Імена — англійською і зі змістом: `price`, а не `x1` чи `tsina`.

PEP 8 · СТИЛЬ

Один код — два вигляди

ПОГАНО

```
def CalcPrice(P,q):
  Res=P*q*1.2
  return Res
```

ДОБРЕ

```
def calc_price(price, qty):
    total = price * qty * 1.2
    return total
```

4 пробіли
відступ, не Tab

≤ 79 символів
довжина рядка

a = b + c
пробіли навколо операторів

2 порожні рядки
між функціями та класами

ПИТАННЯ



НУ ЩО, ЯКІ ВРАЖЕННЯ?

Що було найцікавішим на першому занятті?



СПОЙЛЕР

НАСТУПНОГО ЗАНЯТТЯ

→ **Модуль 2. Керуючі конструкції. Винятки**

ДЯКУЮ

